

SOFTWARE ENGINEERING

Midterm Activity 5

Web-Based Applicant Information and Employment Assistance Management System for the Public Employment Service Office (PESO) of Agoo, La Union

Purpose of the Document

This report identifies portions of the PESO-Web-Based-AIMS project that demonstrate (1) Professional Coding Standards and Practices, (2) the 15 Principles of Quality Software, (3) Defensive Programming, and (4) Error/Exception Handling. It also identifies areas that do not fully implement these practices. Findings are tied to identifiable project files and methods so that the assessment is clear, traceable, and useful for improvement.

1. Professional Coding Standards and Practices

Practice	Code / Project Evidence	Assessment
Framework conventions and separation of concerns	app/Http/Controllers/* separates request/controller logic from app/Models/*. routes/web.php maps routes to controllers.	IMPLEMENTED
Meaningful names	Methods/classes such as updateStatus(), updateApplication(), printApplication(), validateAddressHierarchy(), scopeSearch(), ApplicantManagementController, and EmploymentProgram communicate intent.	IMPLEMENTED
Input validation	PublicApplicantController::store() validates required fields, data types, lengths, e-mail format, database existence, and consent-related fields.	IMPLEMENTED
Documentation/comments	Controllers include method-level descriptions such as handling application submission, updating application status, and printable record views.	IMPLEMENTED
Reusable model behavior	Applicant::scopeSearch(), Application::scopeSearch(), and Application::scopeFilterByProgramCode() centralize reusable query behavior.	IMPLEMENTED
Security-oriented practices	Admin routes use auth middleware; login regenerates the session; passwords are hashed using Laravel's Hash functionality.	IMPLEMENTED
Automated testing	tests/Feature/PublicApplicantSubmissionTest.php uses RefreshDatabase, database seeding, HTTP requests, redirects, and database assertions.	IMPLEMENTED
Style/formatting	Some files are compactly formatted and some imports/logic can be cleaned. PublicApplicantController contains imports that are not used in the reviewed implementation.	NEEDS IMPROVEMENT
Duplication	PublicApplicantController contains similar applicant-field mappings in separate new-applicant and existing-applicant branches.	NEEDS IMPROVEMENT

2. Davis's 15 Principles of Quality Software

The table below deliberately distinguishes source-code evidence from project-level evidence. A principle is not marked fully implemented merely because the software contains related features.

#	Principle	Finding / Evidence	Status
1	Make quality number 1	Validation, authentication, tests, and exception handling show attention to quality, but no formal quality gate is visible.	PARTIAL
2	High-quality software is possible	The project contains validation, tests, modular controllers/models, and logging. Quality is demonstrable but cannot be proven solely by source inspection.	PARTIAL
3	Give products to customers early	The project has a working web application in GitHub; repository history shows incremental development.	PROJECT-LEVEL IMPLEMENTED
4	Determine the problem before writing requirements	The system clearly addresses PESO applicant records, employment programs, application status, reports, and public submission.	PROJECT-LEVEL IMPLEMENTED
5	Evaluate design alternatives	DFD/Structure Chart documentation can support design work, but source code alone does not prove that alternatives were evaluated.	NOT DIRECTLY EVIDENCED
6	Use an appropriate process model	The project context uses Agile, but the repository alone does not prove the complete process was followed.	PARTIAL / PROJECT-LEVEL
7	Use different languages for different phases	PHP/Laravel handles backend logic while JavaScript/CSS/Vite support frontend development.	IMPLEMENTED

8	Minimize intellectual distances	Names such as ApplicantManagementController, EmploymentProgram, updateStatus, and validateAddressHierarchy closely map to business concepts.	IMPLEMENTED
9	Put techniques before tools	Validation, authentication, Eloquent relationships, testing, and logging are techniques; Laravel/Vite are supporting tools.	PARTIAL
10	Get it right before you make it faster	Existence checks, database validation, and automated tests favor correctness. No performance-first optimization is evident.	IMPLEMENTED / PARTIAL
11	Inspect code	Tests and structured validation/error handling exist, but a formal peer code-review checklist or review record is not visible.	IMPLEMENTED / PARTIAL
12	Good management is more important than good technology	This is mainly a management/leadership principle and cannot be established from a code fragment.	NOT PROVABLE FROM CODE
13	People are the key to success	The README identifies five group members, and the repository shows a collaborative project structure/history.	PROJECT-LEVEL IMPLEMENTED
14	Follow with care	Framework conventions are used, but the fixed seeded password and unfinished settings behavior require careful review.	PARTIAL
15	Take responsibility	The team provides documentation, tests, setup files, and activity tracking; remaining limitations should be acknowledged and corrected.	IMPLEMENTED / PARTIAL

3. Defensive Programming

Defensive Technique	Code Evidence	Assessment
Validate external input	PublicApplicantController::store() uses request validation rules for required/nullable values, strings, maximum lengths, dates, e-mail, exists, allowed values, and booleans.	IMPLEMENTED
Validate relationships / consistency	validateAddressHierarchy() checks region, province, city/municipality, and barangay existence and parent-child relationships.	IMPLEMENTED
Guard optional data	The code uses null-coalescing (??), optional chaining (?->), and explicit boolean casts for optional values/flags.	IMPLEMENTED
Fail safely when records do not exist	ProgramModuleController catches ModelNotFoundException and redirects with a user-friendly error message.	IMPLEMENTED
Protect authenticated admin routes	routes/web.php places administrative management routes behind auth middleware.	IMPLEMENTED
Prevent duplicate users	SettingsController validates unique username/e-mail values and hashes passwords before storage.	IMPLEMENTED
Limit query result size	ApplicantManagementController and ProgramModuleController use paginate(10) to avoid unnecessarily large result sets.	IMPLEMENTED
Default credentials	DatabaseSeeder.php creates a default administrator with a fixed demonstration password.	NEEDS IMPROVEMENT
Identifier generation	Applicant/application codes based on Model::count()+1 can collide under concurrent submissions or after deletions.	NEEDS IMPROVEMENT

4. Error / Exception Handling

Code Area	How Errors / Exceptions Are Handled	Assessment
AdminAuthController::login()	Validates login credentials and returns the user to the login page with an error message when authentication fails.	IMPLEMENTED
AdminAuthController::logout()	Checks authentication before activity logging, invalidates the session, and regenerates the CSRF token.	IMPLEMENTED
PublicApplicantController::validateAddressHierarchy()	Throws ValidationException with field-specific messages when the supplied address hierarchy is invalid.	IMPLEMENTED
PublicApplicantController::store()	Uses Laravel validation and ValidationException for invalid application conditions, including missing program-specific requirements.	IMPLEMENTED
ProgramModuleController	Catches ModelNotFoundException for application view/edit/update/delete/print operations and returns user-friendly redirects.	IMPLEMENTED
Laravel validation mechanism	Invalid form input is rejected through the framework validation layer instead of being allowed to proceed unchecked.	IMPLEMENTED

5. Code That Does Not Fully Implement the Required Practices

Code / Area	Observed Limitation
SettingsController::update()	Currently returns a success message without persisting settings. This is a functional completeness gap.
DatabaseSeeder.php	Contains a fixed default administrator password. This is acceptable only as local demonstration data and should not be used as a production credential.
PublicApplicantController.php	Contains duplicated applicant-field mappings between new and existing applicant branches. Refactoring into a reusable method/service/request object would improve maintainability.
PublicApplicantController.php	Unused DB and Validator imports should be removed to improve clarity and cleanliness.
ApplicantManagementController.php	Sensitive update/delete operations appear to rely on authentication middleware; role-level authorization for operations such as deleting applicants is not clearly evidenced.
Applicant/application code generation	count()+1 is vulnerable to race conditions. A UUID/database sequence or retry-safe unique-number strategy would be stronger.
Peer code inspection	Automated tests are present, but a formal code-review checklist, pull-request record, or peer-review report is not directly evidenced.

6. Source Files Reviewed

- routes/web.php — route organization and authentication middleware.
- composer.json — PHP/Laravel requirements and project scripts.
- .env.example — default SQLite configuration.
- app/Http/Controllers/PublicApplicantController.php — validation, defensive checks, and public submission flow.
- app/Http/Controllers/ApplicantManagementController.php — applicant search, pagination, validation, update/delete operations.
- app/Http/Controllers/ProgramModuleController.php — application modules and ModelNotFoundException handling.
- app/Http/Controllers/AdminAuthController.php — authentication, session handling, and activity logging.
- app/Http/Controllers/ReportController.php — report filtering and processing.
- app/Http/Controllers/SettingsController.php — user validation, password hashing, and settings behavior.
- app/Models/Applicant.php and app/Models/Application.php — relationships, scopes, fillable fields, and casts.
- database/seeders/DatabaseSeeder.php — seed data and default administrator credentials.
- tests/Feature/PublicApplicantSubmissionTest.php — feature testing and database assertions.

Overall conclusion: PESO-Web-Based-AIMS demonstrates several strong engineering practices, especially modular Laravel organization, input validation, authentication, defensive checks, pagination, automated testing, and structured exception handling. However, the project should not be described as fully compliant with all software-quality principles. The identified gaps, especially fixed demo credentials, identifier generation, duplicated logic, incomplete settings persistence, and lack of directly evidenced peer review should be addressed or explicitly documented as limitations.